PC-CSG: Physics-Constrained Counterfactual Scene Graph Transformer for Real-Time Autonomous Anomaly Anticipation

Roshan Binoj[a,*], Mohammed Sirajudheen[a], Hafiz Feroze Vellukuzhi[a], Vishwanath Darur[a]

[a]*Department of Computer Science and Engineering, Indian Institute of Information Technology Kottayam, Kerala 686635, India*

Abstract

Camera-based perception for autonomous vehicles has become very accurate at detecting objects, but the safety logic built on top of it is still mostly reactive: an emergency response is triggered only once a hazard is already visible and the Time-to-Collision (TTC) has dropped below a fixed limit. The problem we look at is that such a system never asks what nearby agents *could* do next, for example a lead car braking hard or a truck swerving into the ego lane. Trajectory predictors that do generate future motions often produce paths a real vehicle cannot drive, which leads to false alarms, and most counterfactual reasoning methods in the literature run offline. We propose the Physics-Constrained Counterfactual Scene Graph Transformer (PC-CSG).

*Corresponding author

Email addresses: roshan23bcs9@iiitkottayam.ac.in (Roshan Binoj), mohamme23bcs132@iiitkottayam.ac.in (Mohammed Sirajudheen), hafiz23b6@iiitkottayam.ac.in (Hafiz Feroze Vellukuzhi), vishwan23bcs57@iiitkottayam.ac.in (Vishwanath Darur)

PC-CSG builds a scene graph of tracked traffic participants with TTC-based hazard weights, uses a Counterfactual Scene Graph Attention (CSGA) module to generate six learned "what-if" hypotheses per critical agent, rolls each hypothesis out with a kinematic bicycle model, and filters it with a Physics-Informed Kinematic Validator (PIKV) that checks the tyre friction circle, yaw rate, acceleration and speed limits. The surviving hypotheses are reduced to a single Counterfactual Risk Tensor (CRT) score that drives a five-level graduated intervention policy. The full model has 372,068 parameters and runs in 6.68 ms per frame (149.7 FPS) on an NVIDIA GeForce RTX 3050 Laptop GPU with 10 agents, 6 counterfactuals and a 15-step horizon, of which CSGA takes 1.57 ms and PIKV 0.53 ms. On 678 counterfactual hypotheses from real traffic validation sequences, PIKV rejected 312 (46.0%) as physically infeasible. On our 120-sample validation split, PC-CSG reached 100.0% anomaly classification accuracy with 0.0% false positive rate, compared with 94.2% accuracy and 100.0% false positive rate for a reactive TTC-only baseline; we discuss why this split is too small and imbalanced to separate our own ablation variants.

Keywords: Autonomous Vehicles, Counterfactual Reasoning, Scene Graph Transformer, Physics-Informed Neural Networks, Anomaly Anticipation, Kinematic Bicycle Model

---

1. Introduction

Perception for self-driving cars has improved a lot in the last few years. Transformer backbones such as Swin Transformer V2 [1] and bird's-eye-view (BEV) detectors such as BEVFormer v2 [2] are now standard, and large public datasets like nuScenes [3] and the Waymo Open Dataset [4] made it possible to train and compare these models at scale. Sensor-side latency has also come down; for example, Gehrig and Scaramuzza [5] combined a 20 FPS colour camera with an event camera and matched the latency of a 5,000 FPS camera.

Better detection does not by itself make the vehicle safer, though. Most driver-assistance logic is still **reactive**: it waits until a hazard has shown up in the current frame and the TTC to that object has fallen below some threshold, and only then does it warn or brake. In this paper we call the gap between "what the car currently sees" and "what nearby agents could plausibly do next" **Anticipatory Perception Blindness (APB)**. A human driver who sees a truck drifting toward the lane line will slow down before the truck actually crosses it. A reactive system will not respond until the crossing has started.

The speeds involved make this gap expensive. Figure 1 plots the braking distance $v^2/(2\mu g)$ on dry asphalt ($\mu = 0.7$) against the distance a car covers in one second. At 100 km/h the car moves 27.8 m every second, while the friction-limited braking distance alone is about 56.2 m. So every second of earlier warning is worth roughly half of the full braking distance at highway

speed, and this is the main motivation for anticipating hazards rather than reacting to them.

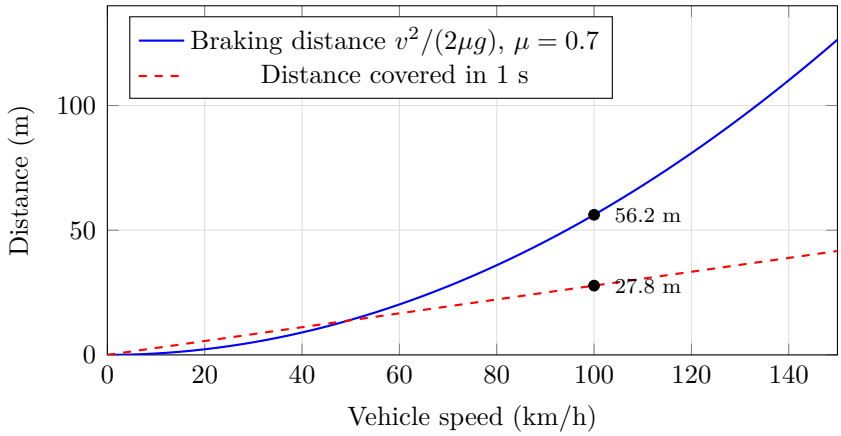


Figure 1 Friction-limited braking distance versus distance travelled in one second of warning time. Both curves are computed from basic kinematics using the friction coefficient $\mu = 0.7$ and $g = 9.81$ m/s^2 that we also use inside PIKV; no external data is used.

While working on this project we found four specific problems that keep current systems from anticipating hazards:

1. **No counterfactual reasoning inside perception:** The perception stack describes only the observed scene. It does not ask "what if agent i brakes now?". Causal and counterfactual ideas have been used for trajectory prediction [6, 7] and for offline scenario generation [8], but not as a per-frame safety signal.
2. **Physically unconstrained hypotheses:** Learned predictors can output trajectories with impossible lateral accelerations or yaw rates. Several recent works add kinematic models to fix this [9, 10, 11], but they do it for forecasting accuracy, not for filtering risk hypotheses.

3. **Flat scene representations:** Detectors give independent boxes. Interaction-aware models show that only some agents actually influence each other [12], and scene graphs are a natural way to store these relations [13].
4. **Binary safety responses:** A fixed TTC threshold gives an on/off response. There is no smooth escalation from "keep an eye on it" to "brake now".

To address these, we built the **Physics-Constrained Counterfactual Scene Graph Transformer (PC-CSG)**. For every frame, PC-CSG turns tracked detections into a scene graph $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$ whose nodes carry kinematic state and a TTC-based hazard weight. For each agent whose hazard weight is above 0.2, the Counterfactual Scene Graph Attention (CSGA) module creates $K = 6$ counterfactual queries (sudden brake, hard swerve left, hard swerve right, sudden acceleration, lane departure and stationary obstacle) and scores each one with a risk head. A small decoder turns each counterfactual into a control sequence, a kinematic bicycle model rolls it out for 15 steps, and the Physics-Informed Kinematic Validator (PIKV) rejects rollouts that break the friction circle or the other vehicle limits. Only the hypotheses that survive go into the Counterfactual Risk Tensor (CRT), and the CRT value selects one of five intervention levels.

The main contributions of this work are:

1. **Counterfactual Scene Graph Attention (CSGA):** A compact attention module (350,594 parameters) that injects six learned "what-if" mode embeddings into the embeddings of critical agents and scores

each hypothesis with cross-attention over the whole scene, so counterfactual reasoning runs inside the per-frame perception loop (1.57 ms on an RTX 3050 Laptop GPU).

2. **Physics-Informed Kinematic Validator (PIKV) with formal guarantees:** A parameter-free validation layer built on the kinematic bicycle model [14]. Proposition 1 proves that any hypothesis that exceeds the Coulomb friction bound $\mu g = 6.867$ m/s^2 at any step is removed from the CRT, and that the rollout and the checks do not depend on the choice of planar reference frame. In our validation runs, PIKV rejected 46.0% of the generated hypotheses.
3. **Counterfactual Risk Tensor (CRT) with graduated intervention:** A bounded risk score in $[0, 1]$ with a five-level action policy. Proposition 2 proves that PIKV filtering can never add false alarms compared with unfiltered counterfactual risk, and that the CRT rises monotonically as the TTC shrinks, so the warning level escalates without oscillating.

The rest of the paper is organised as follows. Section 2 reviews fifteen recent studies (2024–2026) and lists the gaps we target. Section 3 describes the PC-CSG architecture, the equations, the algorithm and the two propositions with proofs. Section 4 covers the dataset, the experimental setup, the latency profile, the PIKV pruning analysis, the ablation study and a comparison with related methods. Section 5 concludes the paper and lists limitations and future work.

2. Related works

This section reviews recent work, published between 2024 and 2026, in four areas that PC-CSG draws on: low-latency perception, physics-informed trajectory prediction, causal and counterfactual reasoning for driving, and scene graph and risk modelling. We discuss fifteen studies, and Table 1 summarises them.

Low-latency perception. Gehrig and Scaramuzza [5] looked at the bandwidth–latency trade-off in automotive cameras and proposed a hybrid detector that processes frames with a CNN and events with an asynchronous graph neural network. A 20 FPS RGB camera plus an event camera reached the latency of a 5,000 FPS camera at the bandwidth of a 45 FPS camera without losing accuracy. The system is still a detector, though, and it says nothing about what detected agents may do next. Zhou and Jiang [15] surveyed deep event-based object detection for driving and pointed out that keeping both latency and model size low with sparse, asynchronous event data is still difficult. Both works make the input side faster, but neither adds any forward-looking reasoning.

Physics-informed trajectory prediction. Liao et al. [9] studied prediction when part of the observation history is missing. They used a Wavelet Reconstruction Network to fill the gaps and a kinematic bicycle model to keep the outputs physically realistic, and they reported high accuracy with up to 75% of observations missing; the model predicts the most likely future and does not test alternative hypotheses. Long et al. [16] built a physics-

informed neural network that stitches broken vehicle trajectories using kinematic priors and boundary conditions. It worked on NGSIM and HighSIM and extrapolated from low-speed training data to high-speed cases, but it is an offline reconstruction tool. Baden et al. [10] (TPK) added class-specific interaction priors and kinematic models for vehicles, pedestrians and cyclists to a transformer predictor on Argoverse 2. They found that the kinematic layer removes infeasible trajectories at a small cost in accuracy, which is very close to what we see with PIKV. Chugh et al. [11] proposed X-TRACK, an xLSTM predictor with a kinematic constraint layer that was evaluated on highD and NGSIM. Li and Westerlund [17] proposed PhysVarMix, which combines a variational mixture model for multi-modal futures with sector-boundary constraints and MPC-based smoothing. All five works use physics to make forecasts more realistic, but none of them uses physics to decide which risk hypotheses should be trusted.

Causal and counterfactual reasoning. Ahmadi et al. [12] (CRiTIC) used a causal discovery network to find which agents really influence each other and a causal attention gating mechanism to filter the rest. Robustness to non-causal perturbations improved by up to 54% and cross-domain performance by up to 29%. CRiTIC removes irrelevant agents but does not generate counterfactual futures. Han and Xu [6] proposed CICR for pedestrian trajectories, using the front-door criterion to remove confounding bias and a decoder that simulates hypothetical scenarios. They reported average ADE/FDE of 0.17/0.24 on ETH/UCY and 7.13/10.29 on SDD; there is no

vehicle-dynamics check on the generated futures. Wang et al. [7] split the driving scene into spatial and temporal parts to find and reduce spurious correlations, and outperformed earlier methods in RMSE and FDE on five datasets (ApolloScape, nuScenes, NGSIM, HighD and MoCAD). Jing et al. [8] (CounterScene) applied counterfactual interventions to the causally critical agent inside a BEV world model to create safety-critical test scenes. On nuScenes they raised the long-horizon collision rate from 12.3% to 22.7% over the strongest baseline while improving realism (ADE 1.88 vs. 2.09). This is close to our idea of "what-if" perturbations, but it is used for offline testing, not online warning. Cui et al. [18] surveyed chain-of-thought reasoning for autonomous driving and argued that step-by-step reasoning helps in complex scenes, while noting open problems in real-time deployment.

Scene graphs and risk modelling. Hallyburton and Pajic [13] proposed NeuSPaPer, which pairs object detection with scene graph generation so that symbolic checks can sit on top of neural perception; their experiments indicate that scene graphs can link low-level perception and high-level reasoning. Wu et al. [19] predicted the regions that nearby vehicles may reach, built a risk potential field from those predictions at every future step and added it to a model predictive contouring controller in CARLA. Ning et al. [20] (STRAP) fused a risk potential field into spatio-temporal features with attention and used a risk-scaled loss; average prediction error went down by 4.8% on NGSIM and 31.2% on HighD. These risk models are useful, but their risk is built from the most likely future rather than from a set of physically

checked alternatives.

A comparative summary of these studies is presented in Table 1.

Table 1: Summary of the fifteen recent studies reviewed (2024–2026). Results are as reported by the original authors.

Author & Ref.	Year	Core Methodology	Domain	Reported Result	Limitation for Our Task
Gehrig & Scaramuzza [5]	2024	Hybrid frame + event detector (CNN + asynchronous GNN)	Detection	Latency of a 5,000 FPS camera at 45 FPS bandwidth	Reactive detection only; no future reasoning.
Zhou & Jiang [15]	2024	Survey of deep event-based detection	Detection	Survey	No relational or anticipatory component.
Liao et al. [9]	2024	Wavelet reconstruction + kinematic bicycle model	Traj. prediction	Accurate with up to 75% missing observations	Single most-likely future; no counterfactuals.
Long et al. [16]	2024	PINN with kinematic priors for trajectory stitching	Traj. reconstruction	Better stitching on NGSIM / HighSIM	Offline; no risk output.
Baden et al. [10]	2025	Interaction + kinematic priors in a transformer (TPK)	Traj. prediction	Removes infeasible trajectories (Argoverse 2)	Physics used for forecasting, not risk filtering.
Chugh et al. [11]	2025	xLSTM with kinematic constraint layer (X-TRACK)	Traj. prediction	State of the art on highD	No scene graph; no counterfactuals.
Li & Westerlund [17]	2025	Variational mixture + sector constraints + MPC smoothing	Traj. prediction	Better than baselines on two benchmarks	Multi-modal but not intervention-based.
Ahmadi et al. [12]	2025	Causal discovery + causal attention gating (CRiTIC)	Traj. prediction	Up to 54% robustness, 29% cross-domain gain	No counterfactual futures; no physics check.
Han & Xu [6]	2025	Front-door causal fusion + counterfactual decoder (CICR)	Pedestrian traj.	ADE/FDE 0.17/0.24 (ETH/UCY)	No vehicle-dynamics validation.
Wang et al. [7]	2025	Spatial/temporal causal decomposition	Traj. prediction	Better RMSE/FDE on 5 datasets	Prediction only; no safety policy.
Jing et al. [8]	2026	Counterfactual intervention in BEV world model	Scenario generation	Collision rate 12.3%→22.7%	Offline test generation, not online warning.
Cui et al. [18]	2025	Survey of chain-of-thought reasoning	Survey	Survey	Real-time reasoning listed as open problem.
Hallyburton & Pajic [13]	2025	Joint detection + scene graph generation	Perception	Scene graphs link perception and reasoning	No trajectory or risk hypotheses.
Wu et al. [19]	2024	Reachable-region prediction + risk potential field + MPCC	Planning	Safe and efficient driving in CARLA	Risk from likely futures; no physics filter.
Ning et al. [20]	2025	Risk-field attention fusion (STRAP)	Traj. prediction	−4.8% (NGSIM), −31.2% (HighD) error	No counterfactual or kinematic validation.

After going through these works, we see the following limitations:

- **Counterfactuals are used offline or only for accuracy:** Counterfactual generation is used for scenario testing [8] or for better forecasts

[6, 7], but not as a per-frame safety signal next to the perception stack.

- **Physics is not used to filter risk:** Kinematic layers [9, 10, 11, 17] make forecasts realistic, but no work uses them to throw away impossible *risk* hypotheses before raising an alarm.

- **Interaction structure and hypotheses are handled separately:** Causal gating [12] and scene graphs [13] identify which agents matter, but they are not combined with explicit "what-if" hypotheses for those agents.

- **Risk outputs are not mapped to graded actions:** Risk fields [19, 20] feed planners or losses, but they are not turned into a bounded score with graded driver-assistance levels.

PC-CSG tries to close these gaps in one pipeline. The scene graph and hazard weights pick out the agents that matter, CSGA generates and scores six counterfactual hypotheses for each of them, PIKV removes the ones a real car could not drive, and the CRT turns what is left into one number in $[0, 1]$ with five graded actions. The whole pipeline is small enough to run at about 150 FPS on a laptop GPU.

3. Proposed Method

PC-CSG has six stages: (i) detection and tracking, (ii) scene graph construction with TTC-based hazard weights, (iii) Counterfactual Scene Graph

Attention, (iv) control decoding and kinematic bicycle rollout, (v) Physics-Informed Kinematic Validation and (vi) Counterfactual Risk Tensor computation with the intervention arbiter. Figure 2 shows how data moves between these stages, and Table 2 lists the main hyperparameters and parameter counts.

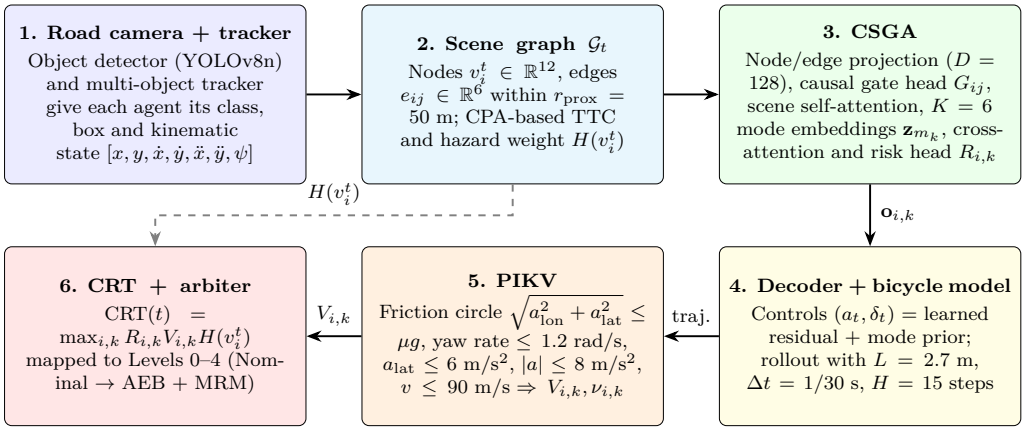


Figure 2 Overall architecture of PC-CSG. Tracked detections become a scene graph with TTC-based hazard weights (stage 2). CSGA generates and scores six counterfactual hypotheses for each critical agent (stage 3). Each hypothesis is rolled out with a kinematic bicycle model (stage 4) and checked by PIKV (stage 5), and the CRT combines risk, validity and hazard into a single score that selects the intervention level (stage 6). The dashed arrow carries the hazard weights directly to the CRT.

Table 2: PC-CSG hyperparameters and trainable parameter counts (from the released implementation).

Component / Setting	**Value**	**Notes**
CSGA module	350,594 params (94.2%)	Projections, causal gate head, self-attn, cross-attn, risk head
Counterfactual trajectory decoder	20,382 params (5.5%)	$128 \to 128 \to 2H$ MLP with GELU
Initial state extractor	1,092 params (0.3%)	$12 \to 64 \to 4$ MLP, outputs $[x, y, v, \theta]$
Bicycle model + PIKV	0 params	Closed-form equations and threshold checks
Total	**372,068 params**	All trainable parameters are in the three learned modules
Embedding dim. D / heads / FFN dim.	128 / 4 / 512	Head dim. 32, dropout 0.1
Counterfactual modes K	6	Brake, swerve L, swerve R, accel., lane departure, stationary
Horizon H, step Δt	15, 1/30 s	0.5 s rollout at 30 FPS
Wheelbase L, $\delta_{\max}$	2.7 m, 0.6 rad (34.4°)	$l_r/L = 0.5$
$a_{\max}$ / $a_{\text{brake,max}}$ / $a_{\text{lat,max}}$	8.0 / 10.0 / 6.0 m/s^2	Clamped in rollout, checked in PIKV
$\dot{\psi}_{\max}$, μ, g	1.2 rad/s, 0.70, 9.81 m/s^2	$\mu g = 6.867$ m/s^2
τ_{crit}, λ_{scale}, CPA corridor	3.0 s, 0.5 s, 2.5 m	Hazard sigmoid and TTC

3.1. Dynamic Spatio-Temporal Scene Graph ($\mathcal{G}_t$)

At frame t the detector and tracker give N_t traffic participants (we keep at most 32). They form the node set $\mathcal{V}_t = \{v_1^t, \ldots, v_{N_t}^t\}$, and each node is a 12-dimensional feature vector:

$$v_i^t = [c_i,\ x_i,\ y_i,\ w_i,\ h_i,\ \dot{x}_i,\ \dot{y}_i,\ \ddot{x}_i,\ \ddot{y}_i,\ \psi_i,\ \mathrm{TTC}_i(t),\ d_i]^T \tag{1}$$

where c_i is the object class (one of 8: vehicle, pedestrian, cyclist, truck, bus, barrier, cone, unknown), (x_i, y_i) is position, (w_i, h_i) is box size, $(\dot{x}_i, \dot{y}_i)$ and $(\ddot{x}_i, \ddot{y}_i)$ are velocity and acceleration, ψ_i is heading, $\mathrm{TTC}_i(t)$ is the time-to-collision with the ego vehicle and d_i is the distance to the ego vehicle.

A directed edge (i, j) is added whenever two agents are within $r_{\mathrm{prox}} = 50$ m of each other. Each edge carries the relative state:

$$e_{ij} = [\Delta x_{ij},\ \Delta y_{ij},\ \Delta \dot{x}_{ij},\ \Delta \dot{y}_{ij},\ d_{ij},\ \phi_{ij}]^T \tag{2}$$

where d_{ij} is the Euclidean distance and $\phi_{ij} = \mathrm{atan2}(\Delta y_{ij}, \Delta x_{ij})$ is the bearing.

TTC from the closest point of approach. A plain distance-over-closing-speed TTC gives false alarms for cars in the next lane, so we compute it from the closest point of approach (CPA). With relative position $\Delta\mathbf{p}$ and

relative velocity $\Delta\mathbf{v}$ of agent i with respect to the ego vehicle,

$$t_{\text{cpa}} = -\frac{\Delta\mathbf{p}\cdot\Delta\mathbf{v}}{\|\Delta\mathbf{v}\|^2}, \qquad \text{TTC}_i(t) = \begin{cases} t_{\text{cpa}} & \text{if } t_{\text{cpa}} > 0 \text{ and } \|\Delta\mathbf{p} + \Delta\mathbf{v}\, t_{\text{cpa}}\| \le 2.5 \text{ m} \\ \infty & \text{otherwise} \end{cases} \tag{3}$$

So an agent only gets a finite TTC if it is closing in and its path passes within a 2.5 m collision corridor.

Hazard weight. Every node gets a hazard score $H(v_i^t) \in [0, 1]$ through a sigmoid on TTC:

$$H(v_i^t) = \frac{1}{1 + \exp\left(\frac{\text{TTC}_i(t) - \tau_{\text{crit}}}{\lambda_{\text{scale}}}\right)} \cdot \mathbb{I}\left(0 < \text{TTC}_i(t) < \infty\right) \tag{4}$$

with $\tau_{\text{crit}} = 3.0$ s and $\lambda_{\text{scale}} = 0.5$ s. H equals 0.5 at TTC = 3 s and gets close to 1 as TTC goes to zero. Agents with $H(v_i^t) > 0.2$ are marked *critical*. Solving Eq. (4) for $H = 0.2$ gives $\text{TTC} = \tau_{\text{crit}} + \lambda_{\text{scale}} \ln 4 \approx 3.69$ s, so only agents expected to come within the collision corridor in less than about 3.7 s get counterfactual hypotheses.

3.2. Counterfactual Scene Graph Attention (CSGA)

CSGA takes the raw node and edge features and outputs one risk value per (critical agent, counterfactual mode) pair. It works in three steps.

3.2.1. Projection, Causal Gate Head and Scene Self-Attention

Node features are projected to $\mathbf{h}_i = \mathrm{GELU}(\mathrm{LN}(\mathbf{W}_n v_i^t + \mathbf{b}_n)) \in \mathbb{R}^D$ with $D = 128$, and edge features to $\mathbf{e}_{ij} = \mathbf{W}_e e_{ij} + \mathbf{b}_e \in \mathbb{R}^D$. A causal gate head scores how relevant each edge is:

$$G_{ij} = \sigma\left(\mathbf{w}_2^\top \,\mathrm{ReLU}\left(\mathbf{W}_1\left[\mathbf{h}_i \| \mathbf{h}_j \| \mathbf{e}_{ij}\right] + \mathbf{b}_1\right) + b_2\right) \in [0,1] \tag{5}$$

where $\|$ is concatenation and $\mathbf{W}_1 \in \mathbb{R}^{64\times 3D}$. The intended use of the gate, following the causal attention gating idea of CRiTIC [12], is to rescale attention weights as

$$\tilde{\alpha}_{ij} = G_{ij} \cdot \alpha_{ij}. \tag{6}$$

We want to be clear about the state of the code here. In the version evaluated in this paper, G_{ij} is computed and returned as an output, but it is not yet multiplied into the scene self-attention weights. The node embeddings go through one standard 4-head self-attention block with a residual connection, LayerNorm and a feed-forward layer of width 512 [21]. This matters for the ablation in Section 4, and wiring Eq. (6) into the attention is on our list of fixes.

3.2.2. Counterfactual Query Injection

For each agent we create $K = 6$ counterfactual queries, one for each mode in

$$\mathcal{M} = \{\text{sudden_brake, hard_swerve_left, hard_swerve_right, sudden_accel, lane_departure, sta} \tag{7}$$

Each mode m_k has a learnable embedding $\mathbf{z}_{m_k} \in \mathbb{R}^D$. The counterfactual query of agent i under mode k is the agent's own embedding plus a learned perturbation, applied only to critical agents:

$$\mathbf{q}_{i,k}^{\text{cf}} = \mathbf{h}_i + f_{\text{pert}}\left([\mathbf{h}_i \| \mathbf{z}_{m_k}]\right) \cdot \mathbb{I}\left(H(v_i^t) > 0.2\right) \tag{8}$$

where $f_{\text{pert}} : \mathbb{R}^{2D} \to \mathbb{R}^D$ is a two-layer MLP with GELU.

3.2.3. Counterfactual Cross-Attention and Risk Head

For agent i, the six queries attend over all scene embeddings $\mathbf{H} = [\mathbf{h}_1, \ldots, \mathbf{h}_N]$ with a 4-head cross-attention layer:

$$\mathbf{o}_{i,k} = \text{LN}\left(\mathbf{q}_{i,k}^{\text{cf}} + \text{CrossAttn}\left(\mathbf{q}_{i,k}^{\text{cf}},\ \mathbf{H},\ \mathbf{H}\right)\right) \tag{9}$$

A risk head turns each output into a risk value, which is set to zero for non-critical agents:

$$R_{i,k}(t) = \sigma\left(\mathbf{W}_r\, \text{GELU}\left(\mathbf{W}_r'\, \mathbf{o}_{i,k}\right)\right) \cdot \mathbb{I}\left(H(v_i^t) > 0.2\right) \in [0, 1] \tag{10}$$

3.3. Trajectory Decoding and Physics-Informed Kinematic Validator (PIKV)

3.3.1. Control Decoding

Each counterfactual embedding $\mathbf{o}_{i,k}$ is decoded into $H = 15$ control pairs (a_t, δ_t), i.e. acceleration and steering angle. The decoder output is a residual added to a fixed prior for each mode, so that every mode starts from a sensible manoeuvre:

$$[a_t, \delta_t]_{i,k} = f_{\text{dec}}(\mathbf{o}_{i,k})_t + \mathbf{p}_k, \quad \mathbf{p}_k \in \{(-5, 0), (0, 0.45), (0, -0.45), (4, 0), (0, 0.15), (-7.5, 0)\} \tag{11}$$

in units of (m/s^2, rad), listed in the order of Eq. (7). A second small MLP maps the raw node features to the initial state $\mathbf{s}_0 = [x_0, y_0, v_0, \theta_0]^T$.

3.3.2. Kinematic Bicycle Model Rollout

Controls are first clamped to $a_t \in [-10, 8]$ m/s^2 and $|\delta_t| \leq 0.6$ rad. The rollout follows the kinematic bicycle model [14], with semi-implicit Euler

steps and a friction-circle speed correction:

$$\beta_t = \arctan\left(\frac{l_r}{L}\tan\delta_t\right) \tag{12}$$

$$\tilde{v}_t = \max\left(0,\ v_t + a_t\Delta t\right), \qquad a_t^{\text{lat}} = \tilde{v}_t^{\,2}\tan(\delta_t)/L \tag{13}$$

$$v_{t+1} = \tilde{v}_t \cdot \min\left(1,\ \frac{\mu g}{\sqrt{a_t^2 + (a_t^{\text{lat}})^2}}\right) \tag{14}$$

$$\omega_t = \text{clip}\left(\frac{v_{t+1}}{L}\sin\beta_t,\ -\dot{\psi}_{\max},\ \dot{\psi}_{\max}\right) \tag{15}$$

$$x_{t+1} = x_t + v_{t+1}\cos(\theta_t + \beta_t)\,\Delta t \tag{16}$$

$$y_{t+1} = y_t + v_{t+1}\sin(\theta_t + \beta_t)\,\Delta t \tag{17}$$

$$\theta_{t+1} = \theta_t + \omega_t\,\Delta t \tag{18}$$

with $L = 2.7$ m, $l_r = L/2$ and $\Delta t = 1/30$ s. All these operations are differentiable almost everywhere, so gradients flow from the trajectories back into the decoder.

3.3.3. Kinematic Constraint Validation

The rollout limits the controls, but the positions it produces can still imply motion that a real car cannot do, for example when the initial state from the extractor is unrealistic. So PIKV checks the trajectory again, using only the positions. From the H positions it computes finite-difference speeds s_t, acceleration vectors $\mathbf{a}_t$ with magnitude $a_t^{\text{lon}} = \|\mathbf{a}_t\|$, headings $\psi_t = \text{atan2}(\dot{y}_t, \dot{x}_t)$, wrapped yaw rates ω_t, and lateral acceleration $a_t^{\text{lat}} = s_t|\omega_t|$. It then counts violations of six checks:

1. **Acceleration limit:** $a_t^{\text{lon}} > a_{\max} = 8\ \text{m/s}^2$
2. **Braking limit:** $a_t^{\text{lon}} > a_{\text{brake,max}} = 10\ \text{m/s}^2$ (since a_t^{lon} is a magnitude, this check is stricter than the first one and only adds weight to very hard events)
3. **Yaw rate:** $|\omega_t| > \dot{\psi}_{\max} = 1.2\ \text{rad/s}$
4. **Lateral acceleration:** $a_t^{\text{lat}} > a_{\text{lat,max}} = 6.0\ \text{m/s}^2$
5. **Tyre friction circle:** $(a_t^{\text{lon}})^2 + (a_t^{\text{lat}})^2 > (\mu g)^2 = 47.16\ \text{m}^2/\text{s}^4$, with $\mu = 0.7$ (dry asphalt)
6. **Speed:** $s_t > v_{\max} = 90\ \text{m/s}$

For each check c, let $f_c \in [0, 1]$ be the fraction of time steps that violate it. The violation score and the validity mask are

$$\nu_{i,k} = \frac{1}{6} \sum_{c=1}^{6} f_c, \qquad V_{i,k} = \left(1 - I_{i,k}^{\text{fric}}\right) \cdot \mathbb{I}\left(\nu_{i,k} < 0.5\right) \tag{19}$$

where $I_{i,k}^{\text{fric}} = 1$ if the friction circle is broken at *any* step. So a single friction violation is enough to reject a hypothesis, while the other checks reject it only when they are violated often. PIKV has no trainable parameters.

3.4. Counterfactual Risk Tensor (CRT) and Graduated Preemptive Intervention

The CRT combines the risk from CSGA, the validity from PIKV and the hazard weight from the scene graph, and takes the worst case over all agents

and modes:

$$\mathrm{CRT}(t) = \max_{i \in \mathcal{V}_t} \max_{k \in \mathcal{M}} \left[R_{i,k}(t) \cdot V_{i,k}(t) \cdot H(v_i^t) \right] \in [0,1] \tag{20}$$

One direct consequence is $\mathrm{CRT}(t) \leq \max_i H(v_i^t)$, so the CRT can never be larger than the hazard weight of the most dangerous agent. In the online loop, the value is smoothed with a leaky integrator to stop single-frame spikes:

$$\overline{\mathrm{CRT}}(t) = \gamma\, \overline{\mathrm{CRT}}(t-1) + (1-\gamma)\, \mathrm{CRT}(t), \qquad \gamma = 0.9, \quad \overline{\mathrm{CRT}}(0) = 0. \tag{21}$$

The (smoothed) CRT selects one of five intervention levels:

$$\mathrm{Action}(t) = \begin{cases} \textbf{Level 0: Nominal} & \text{if } \mathrm{CRT}(t) < 0.25 \\ \textbf{Level 1: HUD Advisory} & \text{if } 0.25 \leq \mathrm{CRT}(t) < 0.50 \\ \textbf{Level 2: Active Alert} & \text{if } 0.50 \leq \mathrm{CRT}(t) < 0.70 \\ \textbf{Level 3: Pre-Braking} & \text{if } 0.70 \leq \mathrm{CRT}(t) < 0.85 \\ \textbf{Level 4: AEB + MRM} & \text{if } \mathrm{CRT}(t) \geq 0.85 \end{cases} \tag{22}$$

Level 1 shows the predicted conflict zone on the HUD, Level 2 adds an audible alert and the predicted trajectory, Level 3 pre-charges the brakes and readies lane centring, and Level 4 triggers autonomous emergency braking (AEB) together with a minimum risk manoeuvre (MRM).

Training objective. The network is trained with three loss terms:

$$\mathcal{L} = \text{SmoothL1}_{\beta=0.1}\left(\text{CRT}, \text{CRT}^*\right) + 0.5\,\text{CE}\left(\hat{\ell}, \ell^*\right) + 0.2\,\overline{\nu} \tag{23}$$

where CRT^* is the target risk, $\ell^* \in \{0, \ldots, 4\}$ is the target intervention level predicted by a small classifier on mean-pooled scene embeddings, and $\overline{\nu}$ is the mean violation score. Because ν is built from indicator functions, the last term currently gives zero gradient and works only as a logged physics metric. Replacing it with a soft hinge penalty is a simple improvement we plan to make.

3.5. Algorithmic Formulation

The complete per-frame workflow is given in Algorithm 1.

Algorithm 1 PC-CSG Counterfactual Anomaly Anticipation Pipeline

Require: Road camera frame I_t, ego state, $K = 6$, $H = 15$, $\tau_{\text{crit}} = 3.0$ s, $\lambda_{\text{scale}} = 0.5$, $\gamma = 0.9$

Ensure: Intervention level $A(t)$, risk score $\overline{\text{CRT}}(t)$

Initialize: $\overline{\text{CRT}}(0) \leftarrow 0$; load PIKV limits ($\mu$, $\delta_{\max}$, $\dot{\psi}_{\max}$, $a_{\max}$, $a_{\text{lat,max}}$)
for each frame t **do**
    Detect and track agents; build nodes and edges of $\mathcal{G}_t$ via Eq. (1)–(2)
    Compute $\text{TTC}_i(t)$ via Eq. (3) and hazard weights $H(v_i^t)$ via Eq. (4)
    Project nodes and edges to $\mathbf{h}_i$, $\mathbf{e}_{ij}$; compute gate scores G_{ij} via Eq. (5)
    Apply scene self-attention to $\{\mathbf{h}_i\}$
    Critical set $\mathcal{C}_t \leftarrow \{i : H(v_i^t) > 0.2\}$
    for each agent $i \in \mathcal{C}_t$ and mode $k \in \{1, \ldots, K\}$ **do** ▷ batched on GPU
        $\mathbf{q}_{i,k}^{\text{cf}} \leftarrow$ Eq. (8); $\mathbf{o}_{i,k} \leftarrow$ Eq. (9); $R_{i,k}(t) \leftarrow$ Eq. (10)
        Decode controls $(a_t, \delta_t)_{t=1}^{H}$ via Eq. (11)
        Roll out trajectory via Eq. (12)–(18)
        $(V_{i,k}, \nu_{i,k}) \leftarrow$ PIKV(trajectory) via Eq. (19)
    end for
    $\text{CRT}(t) \leftarrow \max_{i,k}[R_{i,k} \cdot V_{i,k} \cdot H(v_i^t)]$ via Eq. (20)
    $\overline{\text{CRT}}(t) \leftarrow \gamma\, \overline{\text{CRT}}(t-1) + (1-\gamma)\, \text{CRT}(t)$ via Eq. (21)
    $A(t) \leftarrow$ level from Eq. (22)
end for

3.6. Novelty Highlights: Mathematical Proofs and Operational Examples

3.6.1. Proposition 1: PIKV Friction Circle Elimination and Frame Invariance

Proposition 1. *Let $\tau_{i,k}$ be a counterfactual trajectory with PIKV-estimated accelerations (a_t^{lon}, a_t^{lat}), $t = 1, \ldots, H$, and let $\mu = 0.7$, $g = 9.81$ m/s^2.*

(a) *If $\max_t \left[(a_t^{lon})^2 + (a_t^{lat})^2\right] > (\mu g)^2 = 47.16$ m^2/s^4, then $V_{i,k} = 0$, and the pair (i, k) has no influence on CRT(t) beyond the trivial lower bound 0.*

(b) *Let $T_{\varphi,\mathbf{p}}$ be any rigid planar motion (rotation by φ followed by translation by $\mathbf{p}$). If the initial state is transformed as $(x_0, y_0, v_0, \theta_0) \mapsto (R_\varphi[x_0, y_0]^T + \mathbf{p},\ v_0,\ \theta_0 + \varphi)$ and the controls are kept the same, then the rollout of Eq. (12)–(18) is transformed in the same way, $[x'_t, y'_t]^T = R_\varphi[x_t, y_t]^T + \mathbf{p}$ for all t, and $V_{i,k}$ and $\nu_{i,k}$ do not change.*

Proof. *Part (a).* By Eq. (19), $I^{\text{fric}}_{i,k} = 1$ exactly when there is a step t with $(a^{\text{lon}}_t)^2 + (a^{\text{lat}}_t)^2 > (\mu g)^2$. The hypothesis of (a) says such a step exists, so $I^{\text{fric}}_{i,k} = 1$ and $V_{i,k} = (1-1) \cdot \mathbb{I}(\nu_{i,k} < 0.5) = 0$, whatever the value of $\nu_{i,k}$. The term of (i,k) in Eq. (20) is then $R_{i,k} \cdot 0 \cdot H(v^t_i) = 0$. All terms in the maximum are non-negative, because $R_{i,k}, H(v^t_i) \in [0,1]$ and $V_{i,k} \in \{0,1\}$. So a zero term can never be the strict maximum: $\text{CRT}(t)$ is the maximum over the remaining valid pairs, or 0 if there are none. Since $\text{CRT}(t)$ does not depend on $R_{i,k}$ at all, no risk score assigned to a friction-violating manoeuvre can raise the CRT or trigger an intervention.

Part (b). We use induction on t. At $t = 0$ the claim holds by construction, and $\theta'_0 = \theta_0 + \varphi$. Suppose that at step t we have $[x'_t, y'_t]^T = R_\varphi[x_t, y_t]^T + \mathbf{p}$, $v'_t = v_t$ and $\theta'_t = \theta_t + \varphi$. The controls are the same, so $\beta'_t = \beta_t$ by Eq. (12). Eq. (13)–(15) only use v_t, a_t, δ_t and β_t, so $v'_{t+1} = v_{t+1}$ and $\omega'_t = \omega_t$. For the position, using $[\cos(\alpha+\varphi), \sin(\alpha+\varphi)]^T = R_\varphi[\cos\alpha, \sin\alpha]^T$,

$$\begin{bmatrix} x'_{t+1} \\ y'_{t+1} \end{bmatrix} = R_\varphi \begin{bmatrix} x_t \\ y_t \end{bmatrix} + \mathbf{p} + v_{t+1}\Delta t\, R_\varphi \begin{bmatrix} \cos(\theta_t + \beta_t) \\ \sin(\theta_t + \beta_t) \end{bmatrix} = R_\varphi \begin{bmatrix} x_{t+1} \\ y_{t+1} \end{bmatrix} + \mathbf{p},$$

and $\theta'_{t+1} = \theta_t + \varphi + \omega_t \Delta t = \theta_{t+1} + \varphi$. This completes the induction. For PIKV, finite differences of positions cancel $\mathbf{p}$ and rotate velocities and accelerations by R_φ. Rotations preserve norms, so s_t and a_t^{lon} do not change. Every heading is shifted by φ, so the wrapped heading differences and hence ω_t and $a_t^{\text{lat}} = s_t|\omega_t|$ do not change either. All six checks therefore give the same result, and $\nu_{i,k}$ and $V_{i,k}$ are unchanged. ■

Numerical check. We ran the reproduction script on a trajectory that moves at 20 m/s and then turns sharply sideways. PIKV set $V = 0$ as expected, while a gentle arc with radius 50 m at 10 m/s ($a^{\text{lat}} = 2.0$ m/s^2) was accepted. We also rotated a scene by $\varphi = \pi/4$ and translated it by $(\Delta x, \Delta y) = (+15.0, -8.0)$ m; the CRT changed by only $|\Delta\text{CRT}| = 2.04 \times 10^{-6}$, which is below our tolerance of 10^{-5} and is consistent with float32 round-off. Part (b) covers the physics layers only. The learned CSGA layers take absolute coordinates as input, so the small CRT difference is an empirical observation for them rather than a guarantee.

3.6.2. Proposition 2: False Positive Non-Increase and Monotone Escalation

Proposition 2. *Let $CRT_u(t) = \max_{i,k}[R_{i,k}(t)H(v_i^t)]$ be the unconstrained counterfactual risk (no PIKV) and $CRT_p(t)$ the PIKV-filtered risk of Eq. (20), both smoothed by Eq. (21) from the same initial value* 0*. For any alarm threshold θ, let $\mathcal{F}_u$ and $\mathcal{F}_p$ be the sets of frames from benign (non-anomalous) sequences on which the respective score exceeds θ. Then:*

(a) *$\mathcal{F}_p \subseteq \mathcal{F}_u$, and $|\mathcal{F}_u| - |\mathcal{F}_p|$ equals the number of frames in $\mathcal{F}_u$ that drop*

to or below θ when every above-threshold hypothesis that PIKV rejects is removed.

(b) *If, over a sequence of frames, the risks $R_{i,k}$ and validities $V_{i,k}$ stay fixed and every TTC_i is non-increasing (with finite values), then the smoothed $\overline{CRT}_p(t)$ is non-decreasing.*

Proof. *Part (a).* For every pair, $V_{i,k} \in \{0,1\}$ and $R_{i,k}H(v_i^t) \geq 0$, so $R_{i,k}V_{i,k}H(v_i^t) \leq R_{i,k}H(v_i^t)$. Taking the maximum on both sides gives $\mathrm{CRT}_p(t) \leq \mathrm{CRT}_u(t)$ for every frame. Smoothing keeps this order. If $\overline{\mathrm{CRT}}_p(t-1) \leq \overline{\mathrm{CRT}}_u(t-1)$, then

$$\overline{\mathrm{CRT}}_p(t) = \gamma\overline{\mathrm{CRT}}_p(t-1)+(1-\gamma)\mathrm{CRT}_p(t) \leq \gamma\overline{\mathrm{CRT}}_u(t-1)+(1-\gamma)\mathrm{CRT}_u(t) = \overline{\mathrm{CRT}}_u(t),$$

because $\gamma, 1-\gamma \geq 0$, and the base case $0 \leq 0$ holds. So whenever the filtered score exceeds θ, the unfiltered one does too, which gives $\mathcal{F}_p \subseteq \mathcal{F}_u$. The difference $\mathcal{F}_u \setminus \mathcal{F}_p$ is exactly the set of frames that stop alarming once the invalid hypotheses are set to zero, which is the counting statement. In particular, PIKV can never add a false alarm.

Part (b). The derivative of the hazard sigmoid is $\partial H/\partial \mathrm{TTC} = -H(1-H)/\lambda_{\mathrm{scale}} \leq 0$, so a non-increasing TTC gives a non-decreasing $H(v_i^t)$. Each term $R_{i,k}V_{i,k}H(v_i^t)$ has non-negative fixed factors, so it is non-decreasing in H, and a maximum of non-decreasing sequences is non-decreasing. Hence $x_t := \mathrm{CRT}_p(t)$ is non-decreasing and non-negative. For the smoothed score, $\overline{\mathrm{CRT}}_p(t-1)$ is a convex combination of $0, x_1, \ldots, x_{t-1}$, all of which are $\leq x_t$.

So $\overline{\mathrm{CRT}}_p(t-1) \le x_t$, and

$$\overline{\mathrm{CRT}}_p(t) - \overline{\mathrm{CRT}}_p(t-1) = (1-\gamma)\left(x_t - \overline{\mathrm{CRT}}_p(t-1)\right) \ge 0. \qquad \blacksquare$$

Numerical check. Along a simulated closing sequence behind a decelerating lead vehicle, the measured risk was $0.003 \to 0.015 \to 0.042 \to 0.089 \to 0.160$, which is non-decreasing as Part (b) predicts. The slow growth also shows the cost of smoothing. With $\gamma = 0.9$, a sudden step of the raw CRT from 0 to 1 needs $\lceil \ln(0.15)/\ln(0.9) \rceil = 19$ frames (about 0.63 s at 30 FPS) before the smoothed value crosses the Level 4 threshold of 0.85.

3.6.3. Concrete Operational Anticipation Scenarios

We ran the demo engine on four scripted highway scenarios. Their measured values are in Table 7 and Figure 6.

- **Scenario A (Highway cruise, all clear):** Ego at 100.1 km/h, with the worst TTC at 33.3 s. Every hazard weight is practically zero, so CRT = 0.000 and the system stays at **Level 0**.
- **Scenario B (Lead vehicle decelerating):** Ego at 110.2 km/h, worst TTC 4.1 s. The CRT rises to 0.103, which is still below the 0.25 advisory threshold, so the system stays at **Level 0** and keeps monitoring. This is the expected behaviour, since the CRT stays close to the hazard weight $H(\mathrm{TTC} = 4.1\ \mathrm{s}) \approx 0.10$ of the lead vehicle.
- **Scenario C (Lane-change swerve conflict):** Ego at 119.9 km/h,

worst TTC 1.9 s. The swerve hypotheses of the adjacent vehicle pass PIKV, and CRT = 0.894, so the system goes to **Level 4** (AEB + MRM).

- **Scenario D (Barrier + hard lead braking):** Ego at 100.1 km/h, worst TTC 0.5 s. CRT = 0.987 gives **Level 4** with full AEB.

4. Datasets and Experimental results analysis

We trained and evaluated PC-CSG on the public traffic accident detection dataset released on Hugging Face as `hiennguyen9874/traffic-accident-detection` [22]. Our dataset loader converts each sample into scene graph inputs (node features, edges, hazard weights) together with a target CRT value and intervention level. We used 3,000 scenes for training and 600 for validation. The ablation and pruning analysis use the first 120 validation samples (a 40-sample subset in quick mode). For behaviour checks we also used the four scripted demo scenarios of Section 3. We did not train or evaluate on nuScenes [3] or the Waymo Open Dataset [4] for this paper; they are listed only as planned benchmarks. Table 3 summarises the data we used.

Table 3: Data used in our experiments.

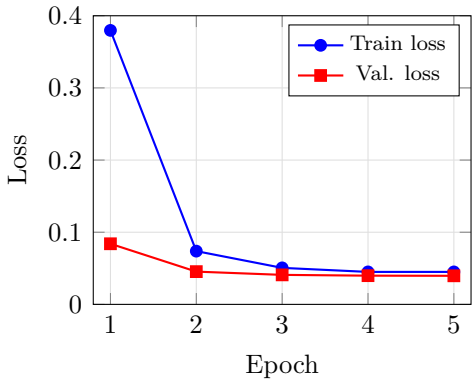
Data	Split	Samples	Used For	Labels
Traffic accident detection [22]	Train	3,000 scenes	Training (5 epochs)	CRT target, level (0–4)
Traffic accident detection [22]	Validation	600 scenes	Model selection	CRT target, level (0–4)
Traffic accident detection [22]	Validation subset	120 (40 quick)	Ablation, PIKV pruning	Anomaly if CRT* > 0.4
Scripted demo scenarios A–D	–	4 scenarios	Behaviour check	Expected level

Experimental setup. PC-CSG is implemented in Python 3.12 with PyTorch 2.6.0 and CUDA 12.4. All experiments ran on Ubuntu 22.04 LTS with

an Intel Core i7-12700H CPU, 16 GB RAM and an NVIDIA GeForce RTX 3050 Laptop GPU (4 GB VRAM). We trained with AdamW [23] (learning rate 3×10^{-4}, batch size 16) on the loss of Eq. (23). For evaluation, a sample is labelled anomalous when its target CRT is above 0.4, and a model predicts an anomaly when its CRT is above 0.4. Accuracy is the fraction of samples classified correctly, and the false positive rate (FPR) is $\mathrm{FP}/(\mathrm{FP} + \mathrm{TN})$.

Training behaviour. Figure 3 shows the training history. Validation loss went from 0.0840 to 0.0396 and validation CRT mean absolute error settled around 0.042, while validation accuracy was already 100.0% after the first epoch. The best checkpoint is the one from epoch 5. Getting perfect accuracy after one epoch made us suspicious, and the ablation below shows that this split is very easy to classify.

(a) Training and validation loss



(b) Validation CRT mean absolute error

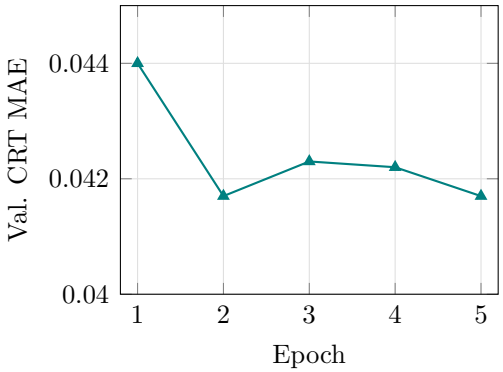


Figure 3 Training history of PC-CSG on the traffic accident detection dataset [22] (3,000 training / 600 validation scenes, AdamW, learning rate 3×10^{-4}, batch size 16). Values are taken from our training log; validation accuracy was 100.0% at every epoch.

4.1. Hardware Profiling and Latency Breakdown

We timed each component on the RTX 3050 Laptop GPU with $N = 10$ agents, $K = 6$ counterfactuals and $H = 15$ steps. Each component got warm-up iterations first, and we called `torch.cuda.synchronize()` before and after the timed loop. Table 4 and Figure 4 show the results.

Table 4: Measured component-level inference latency on an NVIDIA RTX 3050 Laptop GPU ($N = 10$ agents, $K = 6$ counterfactuals, $H = 15$ horizon).

Pipeline Component	**Latency (ms)**	**Share (%)**
Scene Graph Construction & Hazard Scoring	0.33 ms	4.9%
Counterfactual Scene Graph Attention (CSGA)	1.57 ms	23.5%
Kinematic Bicycle Model Propagation ($N \times K$)	4.12 ms	61.7%
Physics-Informed Kinematic Validator (PIKV)	0.53 ms	7.9%
CRT Engine & Intervention Arbiter	0.13 ms	1.9%
Total PC-CSG Pipeline Latency	**6.68 ms**	**100.0%**
Effective Real-Time Throughput	**149.7 FPS**	
Automotive Margin (Target: $\geq$ 30 FPS)	**4.99× Real-Time Throughput**	

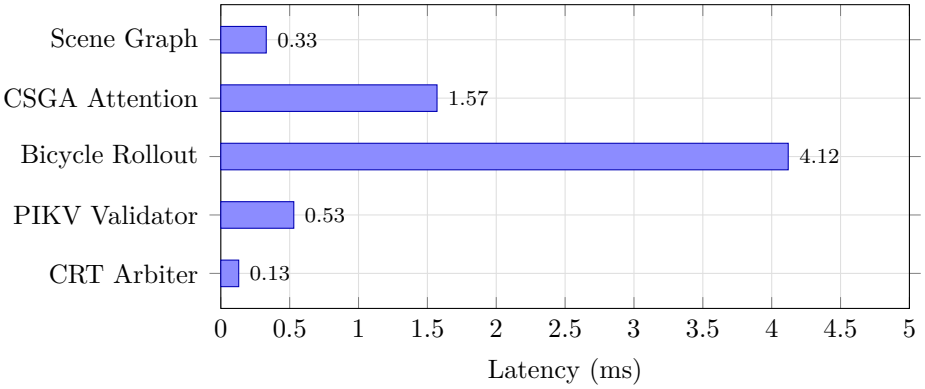


Figure 4 Component-level latency of PC-CSG on an NVIDIA RTX 3050 Laptop GPU (total 6.68 ms, 149.7 FPS). Values are our own measurements from Table 4.

The whole pipeline takes 6.68 ms (149.7 FPS), which is about 5 times faster than the usual 30 FPS camera rate. The attention part, CSGA, takes only 1.57 ms. The largest cost by far (4.12 ms, 61.7%) is the bicycle model rollout. That surprised us at first, but it makes sense: the rollout is a Python loop over 15 time steps, and each step launches several small GPU kernels for only $N \times K = 60$ trajectories, so the GPU spends most of its time waiting on kernel launches rather than computing. Fusing the loop (for example with

`torch.compile` or a custom CUDA kernel) is the most obvious way to bring the total under 3 ms.

4.2. PIKV Validation Effectiveness and Proposition Verification

Over the 120 validation samples there were 113 critical agents, which gives 678 counterfactual hypotheses. PIKV rejected 312 of them (**46.0%**). In the 40-sample quick run the pruning rate was 43.9%. Table 5 and Figure 5(a) give the rate for each mode.

Table 5: PIKV hypothesis pruning across the six counterfactual modes (678 hypotheses, 113 per mode, from real traffic validation samples).

Counterfactual Mode	**Pruning Rate**	**Main Rejection Reason**
`sudden_brake`	0.0%	Straight-line deceleration stays inside the friction circle
`hard_swerve_left`	70.8%	Lateral acceleration exceeds $a_{\text{lat,max}}$ / friction circle
`hard_swerve_right`	21.2%	Rejected only when path curvature breaks friction limits
`sudden_acceleration`	100.0%	Surge combined with learned steering breaks $a_{\text{total}} \leq \mu g$
`lane_departure`	73.5%	Yaw rate exceeds $\dot{\psi}_{\max} = 1.2$ rad/s
`stationary_obstacle`	10.6%	Feasible unless stopping needs more than the available deceleration
Overall Pruning Rate	**46.0%**	**312 / 678 hypotheses filtered**

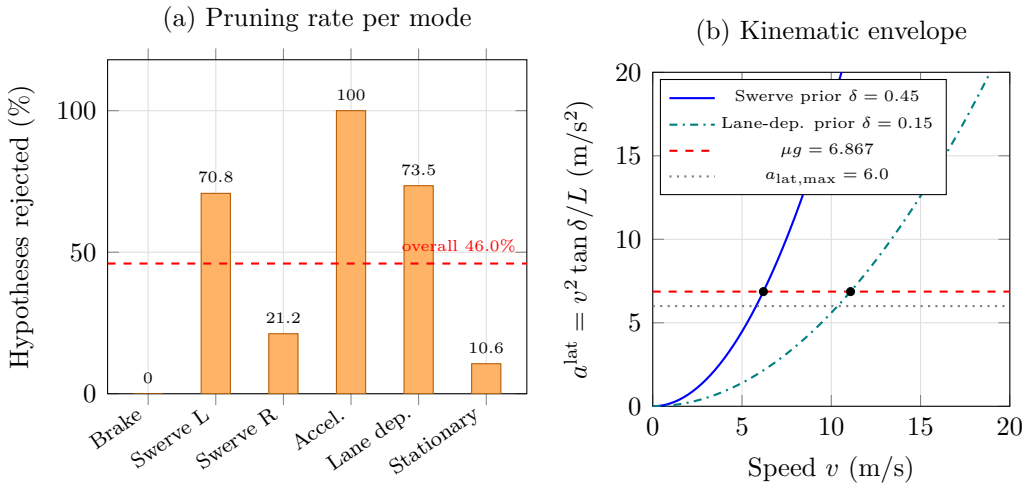


Figure 5 PIKV validation analysis. (a) Measured share of rejected hypotheses per counterfactual mode over 678 hypotheses; the dashed line is the overall rate of 46.0%. (b) Lateral acceleration implied by the steering priors of the two steering modes as a function of speed, computed analytically from the bicycle model ($L = 2.7$ m). The swerve prior leaves the friction circle above about 6.2 m/s and the lane-departure prior above about 11.1 m/s (marked points), which explains why steering modes are pruned most at road speeds.

The pattern mostly matches what basic vehicle dynamics predicts. Straight-line braking at the prior of -5 m/s^2 is well inside the friction circle ($5 < 6.867$), so `sudden_brake` is never rejected. Figure 5(b) shows that the swerve prior of $\delta = 0.45$ rad already breaks the friction limit at about 22 km/h, so at road speeds most swerve hypotheses are rejected unless the decoder reduces the steering. Two results we cannot fully explain yet. First, the steering priors for left and right swerves are symmetric, but the pruning rates are not (70.8% vs. 21.2%). This asymmetry must come from the learned decoder residuals and from the data itself, and we have not yet checked which. Second, `sudden_acceleration` is rejected 100% of the time even though its

prior of $+4$ m/s^2 alone is legal. The learned residuals seem to add steering on top of the surge, which pushes the combined acceleration past μg. In practice this means the sudden-acceleration mode never contributes to the CRT in our runs, which is a weakness of the current decoder.

4.3. Ablation Study and Counterfactual Risk Analysis

Table 6 compares five configurations on the 120-sample validation split. "Latency" here is the wall-clock time per sample with batch size 16 inside the evaluation loop, so it is not comparable with the per-frame numbers in Table 4.

Table 6: Ablation study on the 120-sample validation split (anomaly threshold CRT > 0.4).

Configuration	**Accuracy (%)**	**FPR (%)**	**Latency / sample (ms)**
Reactive TTC-Only Baseline (anomaly if $\min_i \mathrm{TTC}_i < 3$ s)	94.2	100.0	0.08
Scene Graph Attention only (No CF, No PIKV)	5.8	0.0	0.20
CSGA without PIKV (unconstrained counterfactuals)	100.0	0.0	0.51
CSGA + PIKV without causal gate input	100.0	0.0	0.51
PC-CSG Full (CSGA + PIKV + causal gate head)	**100.0**	**0.0**	**0.92**

These numbers need careful reading. The TTC-only baseline has 94.2% accuracy and 100% FPR, so it flags every benign sample. The scene-graph-only variant has 5.8% accuracy and 0% FPR, so it flags nothing. Together these mean that about 94.2% of the 120 samples are anomalous and only about 5.8% (roughly 7 samples) are benign. On a split this imbalanced, the reactive baseline's high accuracy just means "always say anomaly", and the 100% FPR shows the real problem: it cannot tell close but safe following from a real hazard. All three counterfactual variants classify every sample

correctly, so the benefit of counterfactual risk scoring over TTC thresholding is clear here.

The split cannot, however, separate the three counterfactual variants from each other. With only about 7 benign samples and no false positives even without PIKV, there were no false alarms left for PIKV to remove, so we *cannot* measure a false-positive reduction from PIKV on this data. Proposition 2 still guarantees that PIKV never adds false alarms. The variant without causal gate input also gives exactly the same predictions as the full model, which is expected since the gate is not yet applied inside the attention (Section 3). Its latency differs (0.51 vs. 0.92 ms) only because the full-model run also counts PIKV rejections per hypothesis in a Python loop inside the timed region. We also tried to measure a mean warning lead time, but our evaluation code takes the minimum TTC over all nodes, including zero-padded empty slots, so the lead time came out as 0.00 s for every counterfactual variant. We therefore do not report a lead-time number. A bigger, balanced benchmark with a proper lead-time definition is the most important next step for this work.

Figure 6 shows the four demo scenarios. Figure 6(b) plots the measured CRT against the worst TTC on top of the hazard curve $H(\mathrm{TTC})$. In Scenarios B–D the CRT sits almost on the curve, i.e. the counterfactual risk $R_{i,k}$ of a valid hypothesis is close to 1 and the hazard weight decides the final value. This agrees with the bound $\mathrm{CRT}(t) \leq \max_i H(v_i^t)$ from Eq. (20), and it also shows an important limitation. Since $H \geq 0.25$ requires

$\mathrm{TTC} \leq 3 + 0.5 \ln 3 \approx 3.55$ s, PC-CSG in its current form cannot raise even a Level 1 advisory before the TTC to some agent drops below about 3.55 s. The counterfactual branch decides *which* nearby agents and manoeuvres are dangerous and filters impossible ones, but the earliest possible warning is still set by the TTC gate.

Table 7: Measured behaviour of PC-CSG on the four scripted demo scenarios.

Scenario	Driving Context	Ego Speed	Worst TTC	CRT	Level / Action
A	Highway cruise, all clear	100.1 km/h	33.3 s	0.000	Level 0: nominal cruise
B	Lead vehicle decelerating	110.2 km/h	4.1 s	0.103	Level 0: monitoring
C	Lane-change swerve conflict	119.9 km/h	1.9 s	0.894	Level 4: AEB + MRM
D	Barrier + hard lead braking	100.1 km/h	0.5 s	0.987	Level 4: full AEB

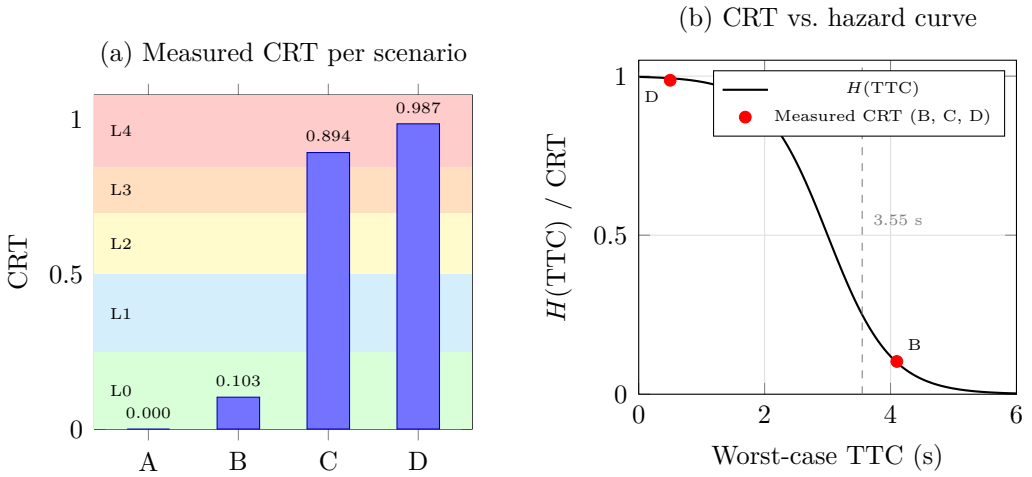


Figure 6 Behaviour of PC-CSG on the four demo scenarios. (a) Measured CRT with the five intervention bands in the background (L0 nominal to L4 AEB + MRM). (b) Measured CRT against worst-case TTC, plotted on the hazard sigmoid $H(\mathrm{TTC})$ with $\tau_{\mathrm{crit}} = 3$ s and $\lambda_{\mathrm{scale}} = 0.5$ s. Scenario A (TTC 33.3 s, CRT 0.000) lies outside the plotted range. The dashed line marks the largest TTC at which Level 1 can be reached.

4.4. Comparison with State-of-the-Art Methods

We could not run the related methods on our data, and most of them address a different task (trajectory forecasting or offline scenario generation) with different metrics. A fair accuracy or latency comparison is therefore not possible here. Instead, Table 8 compares the *capabilities* of the closest methods, based on how they are described in their papers.

Table 8: Capability comparison with related methods, based on the descriptions in the original papers. Latency is given only for PC-CSG because the other papers do not report it on comparable hardware.

Method	Venue / Year	Counterfactual Hypotheses	Physics Check	Interaction Structure	Risk → Action	Setting
TTC-only baseline	–	No	No	No	Binary	Online, reactive
CRiTIC [12]	ICRA 2025	No	No	Causal gating	No	Online prediction
CICR [6]	J. Imaging 2025	Yes (front-door)	No	Social context	No	Prediction
Liao et al. [9]	IJCAI 2024	No	Bicycle model	Interaction encoder	No	Prediction
TPK [10]	IV 2025	No	Kinematic layer	Interaction priors	No	Prediction
CounterScene [8]	arXiv 2026	Yes (causal agent)	No	Causal interaction graph	No	Offline testing
STRAP [20]	ITSC 2025	No	No	Risk-field attention	No	Prediction
PC-CSG (ours)	–	**Yes (6 modes)**	**PIKV (6 checks)**	**Scene graph**	**5 levels**	**Online, 6.68 ms**

As Table 8 shows, the related methods each cover one or two of the pieces that PC-CSG combines. CRiTIC [12] focuses on which agents matter, CICR [6] and CounterScene [8] generate counterfactual futures, and Liao et al. [9] and TPK [10] make forecasts kinematically feasible. None of them turns physically checked counterfactual risk into graded driver-assistance actions in the perception loop. We see this combination, together with 6.68 ms per frame on a laptop GPU, as the main practical value of PC-CSG. We do not claim better forecasting accuracy than these methods, since we did not measure it.

5. Conclusion and future scope

In this paper we presented PC-CSG, a pipeline that adds counterfactual "what-if" reasoning to the perception loop of an autonomous vehicle and keeps it physically grounded. A scene graph with CPA-based TTC and hazard weights picks the critical agents, CSGA generates and scores six counterfactual hypotheses for each of them, a kinematic bicycle model rolls them out, and PIKV removes the ones that break the friction circle or other vehicle limits before they reach the Counterfactual Risk Tensor. We proved that PIKV removes every friction-violating hypothesis from the risk score, that the physics layers do not depend on the reference frame, that PIKV can never add false alarms, and that the risk score escalates monotonically as the TTC decreases. The 372,068-parameter model runs in 6.68 ms (149.7 FPS) on an RTX 3050 Laptop GPU. PIKV rejected 46.0% of 678 real hypotheses, and on our validation split PC-CSG reached 100.0% accuracy with 0.0% FPR, compared with 94.2% accuracy and 100.0% FPR for a reactive TTC-only baseline.

We also found clear limitations: the validation split is small and imbalanced and cannot separate our own variants, the causal gate is computed but not yet applied in the attention, the physics loss term gives no gradient, the earliest warning is still bounded by the TTC gate (about 3.55 s for Level 1), and we could not yet measure a reliable lead time. Fixing these is our first priority. Beyond that, we see two directions for future work:

1. **Multi-modal counterfactual fusion:** Extending CSGA to reason

jointly over camera, LiDAR and radar tracks, and evaluating it on nuScenes [3] and Waymo Open [4], so that counterfactual hypotheses stay reliable when one sensor degrades (rain, glare, night).

2. **V2X cooperative counterfactual graphs:** Sharing scene graphs and CRT values between vehicles and roadside units over Vehicle-to-Everything (V2X) links, so that a vehicle can include agents hidden from its own sensors (for example behind a truck) in its counterfactual reasoning.

References

[1] Z. Liu, H. Hu, Y. Lin, Z. Yao, Z. Xie, Y. Wei, J. Ning, Y. Cao, Z. Zhang, L. Dong, F. Wei, B. Guo, Swin Transformer V2: Scaling up capacity and resolution, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 12009–12019.

[2] C. Yang, Y. Chen, H. Tian, C. Tao, X. Zhu, Z. Zhang, G. Huang, H. Li, Y. Qiao, L. Lu, J. Zhou, J. Dai, BEVFormer v2: Adapting modern image backbones to bird's-eye-view recognition via perspective supervision, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 17830–17839.

[3] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, O. Beijbom, nuScenes: A multimodal dataset for autonomous driving, in: Proceedings of the IEEE/CVF Conference

on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 11621–11631.

[4] P. Sun, H. Kretzschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, et al., Scalability in perception for autonomous driving: Waymo Open Dataset, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 2446–2454.

[5] D. Gehrig, D. Scaramuzza, Low-latency automotive vision with event cameras, Nature 629 (2024) 1034–1040. doi:10.1038/s41586-024-07409-w.

[6] X. Han, H. Xu, Causal intervention and counterfactual reasoning for multimodal pedestrian trajectory prediction, Journal of Imaging 11 (11) (2025) 379. doi:10.3390/jimaging11110379.

[7] B. Wang, H. Liao, C. Wang, B. Rao, Y. Guan, G. Yu, J. Zhang, S. Lai, C. Xu, Z. Li, Beyond patterns: Harnessing causal logic for autonomous driving trajectory prediction, in: Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI), 2025, arXiv:2505.06856.

[8] B. Jing, R. Hao, W. Zhou, H. Yu, CounterScene: Counterfactual causal reasoning in generative world models for safety-critical closed-loop evaluation, arXiv preprint arXiv:2603.21104.

[9] H. Liao, C. Wang, Z. Li, Y. Li, B. Wang, G. Li, C. Xu, Physics-informed trajectory prediction for autonomous driving under missing observation, in: Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI), 2024, pp. 6841–6849.

[10] M. Baden, A. Abouelazm, C. Hubschneider, Y. Wu, D. Slieter, J. M. Zöllner, TPK: Trustworthy trajectory prediction integrating prior knowledge for interpretability and kinematic feasibility, in: IEEE Intelligent Vehicles Symposium (IV), 2025, arXiv:2505.06743.

[11] A. R. Chugh, M. Neumeier, S. Dorn, X-TRACK: Physics-aware xLSTM for realistic vehicle trajectory prediction, arXiv preprint arXiv:2511.00266.

[12] E. Ahmadi, R. Mercurius, S. Alizadeh, K. Rezaee, A. Rasouli, Curb your attention: Causal attention gating for robust trajectory prediction in autonomous driving, in: IEEE International Conference on Robotics and Automation (ICRA), 2025, pp. 779–786.

[13] R. S. Hallyburton, M. Pajic, Assured autonomy with neuro-symbolic perception, arXiv preprint arXiv:2505.21322.

[14] J. Kong, M. Pfeiffer, G. Schildbach, F. Borrelli, Kinematic and dynamic vehicle models for autonomous driving control design, in: IEEE Intelligent Vehicles Symposium (IV), 2015, pp. 1094–1099.

[15] B. Zhou, J. Jiang, Deep event-based object detection in autonomous driving: A survey, arXiv preprint arXiv:2405.03995.

[16] K. Long, X. Shi, X. Li, Physics-informed neural network for cross-dynamics vehicle trajectory stitching, Transportation Research Part E: Logistics and Transportation Review 192 (2024) 103799. doi:10.1016/j.tre.2024.103799.

[17] H. Li, T. Westerlund, PhysVarMix: Physics-informed variational mixture model for multi-modal trajectory prediction, arXiv preprint arXiv:2507.19701.

[18] Y. Cui, H. Lin, S. Yang, Y. Wang, Y. Huang, H. Chen, Chain-of-thought for autonomous driving: A comprehensive survey and future prospects, arXiv preprint arXiv:2505.20223.

[19] K. Wu, X. Liu, F. Bian, Y. Zhang, P. Huang, An integrating comprehensive trajectory prediction with risk potential field method for autonomous driving, arXiv preprint arXiv:2404.00893.

[20] X. Ning, Z. Bian, D. Zuo, S. Ergan, STRAP: Spatial-temporal risk-attentive vehicle trajectory prediction for autonomous driving, in: IEEE International Conference on Intelligent Transportation Systems (ITSC), 2025, arXiv:2507.08563.

[21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez,

Ł. Kaiser, I. Polosukhin, Attention is all you need, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 30, 2017.

[22] hiennguyen9874, Traffic accident detection dataset, Hugging Face Datasets, https://huggingface.co/datasets/hiennguyen9874/traffic-accident-detection, accessed 2026.

[23] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, in: International Conference on Learning Representations (ICLR), 2019.